📧 Email / SMS Spam Classifier using LSTM & BiLSTM

📝 Project Overview

This project implements an **AI-based spam detection system** to classify text messages (emails or SMS) as **Spam** or **Ham (Not Spam)** using **Deep Learning**.

The system leverages:

- **LSTM (Long Short-Term Memory)** networks
- **Bidirectional LSTM (BiLSTM)** for improved context understanding

It demonstrates how sequence models can detect spam patterns in natural language text.

🔹 Dataset

- The dataset contains **text messages** and **labels**:

Feature	Description
`message`	The content of the email or SMS
`label`	0 = Ham (Not Spam), 1 = Spam

- Example messages:

"Free entry in 2 a wkly comp to win FA Cup final tkts" → Spam
"Ok lar... Joking wif u" → Ham

- Preprocessing steps included:
  - Lowercasing
  - Removing punctuation
  - Tokenization
  - Padding sequences to equal length

🛠️ Technologies & Libraries Used

- **Python**
- **TensorFlow / Keras**
- **NumPy / Pandas**
- **Scikit-Learn**
- **Matplotlib / Seaborn** (for visualizations, optional)

Key Keras modules:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Bidirectional
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping
```

🧠 Model Architecture

1️⃣ LSTM Model

- Embedding layer → LSTM → LSTM → Dense (Sigmoid)
- Dropout used for regularization
- Output: **Probability of Spam**

2️⃣ BiLSTM Model

- Embedding layer → **Bidirectional LSTM** → Dense (Sigmoid)
- Reads text in **both forward and backward directions**
- Captures full context of the message

```
model2 = Sequential()
model2.add(Embedding(vocab_size, embedding_dim, input_length=max_len))
model2.add(Bidirectional(LSTM(n_lstm, dropout=drop_lstm)))
model2.add(Dense(1, activation='sigmoid'))
model2.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

🔍 Training

- Loss function: **Binary Crossentropy**
- Optimizer: **Adam**
- Metric: **Accuracy**
- EarlyStopping used to prevent overfitting:

```
early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
```

- Trained on padded sequences of tokenized text messages.

📈 Evaluation Metrics

- **Accuracy**: Percentage of correctly classified messages
- **Precision / Recall / F1 Score** (optional for detailed analysis)

Example predictions:

Message	Model Output (Probability)	Prediction
"Call me"	0.0714	Ham
"You are awarded a Nikon Camera"	0.8804	Spam

🚀 Usage / Deployment

You can deploy the classifier as a **real-time prediction tool**:

```
def predict_spam(messages):
    sequences = tokenizer.texts_to_sequences(messages)
    padded = pad_sequences(sequences, maxlen=max_len, padding='post')
    preds = model.predict(padded)
    results = ["Spam" if p > 0.5 else "Ham" for p in preds]
    return results
```

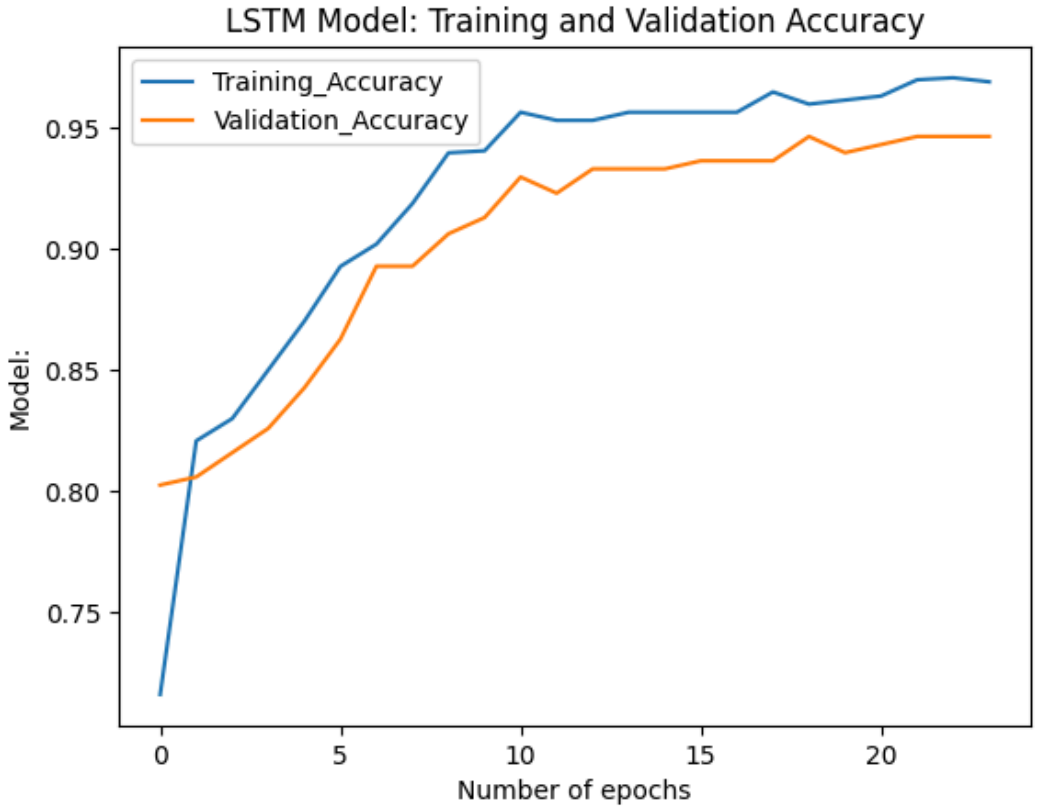
- Input: List of text messages
- Output: List of predictions (`Spam` / `Ham`)

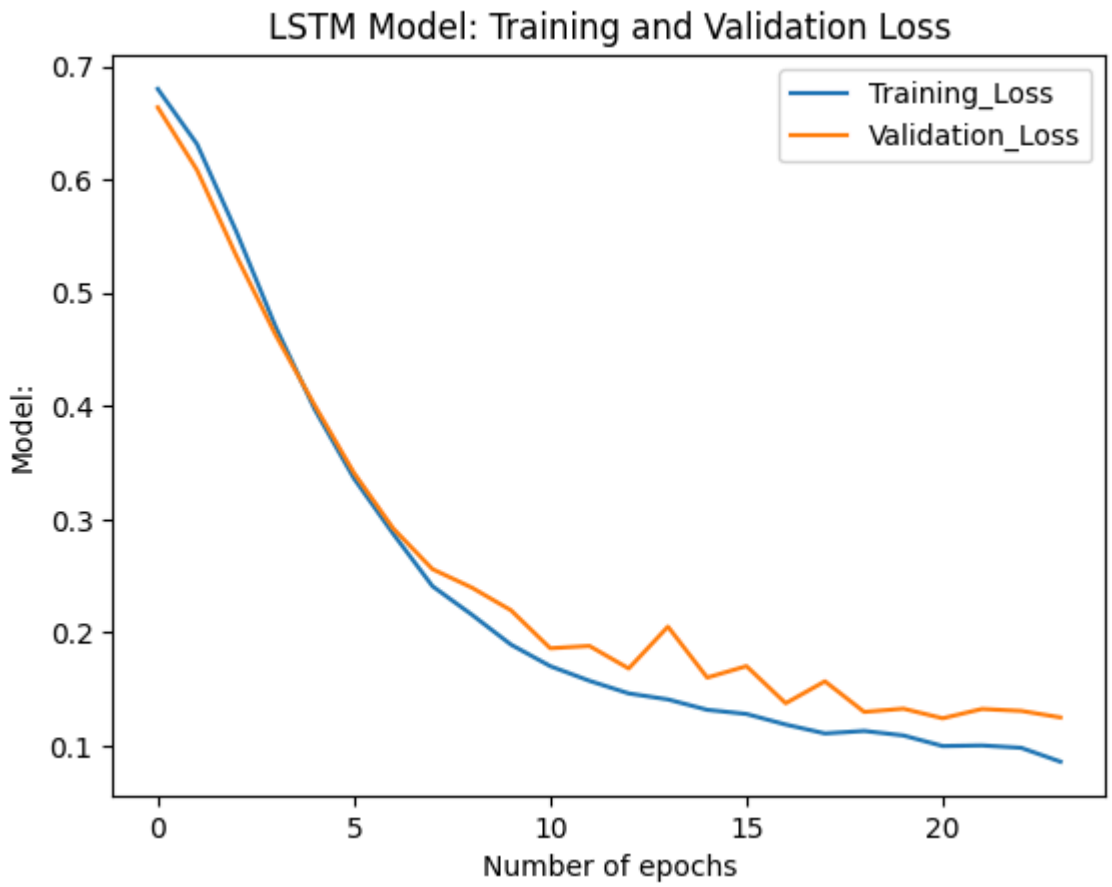
You can also integrate it with **Gradio or Streamlit** for an interactive web interface.

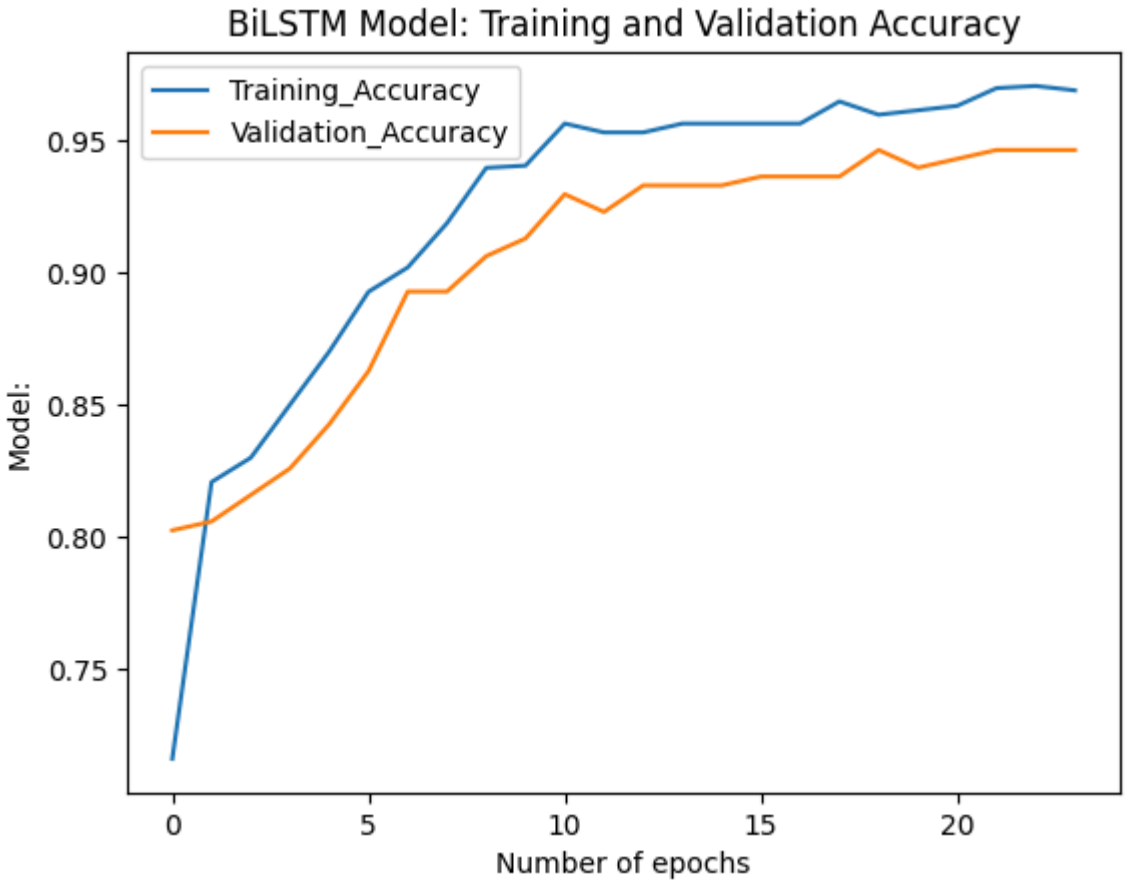
🔹 Key Learnings

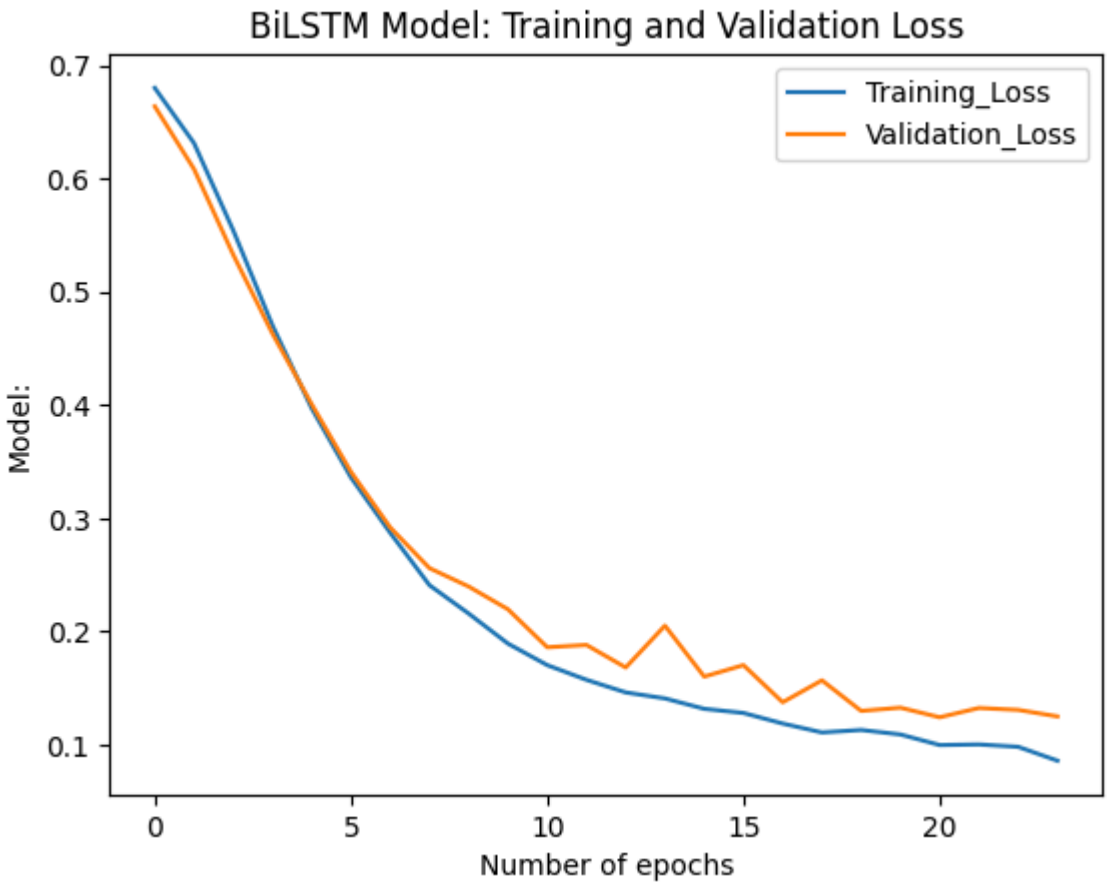
- Text preprocessing and sequence padding are crucial for LSTM models
- LSTM captures sequential dependencies; BiLSTM captures **forward + backward context**
- Neural networks output probabilities, which can be thresholded for classification
- EarlyStopping prevents overfitting and ensures best model weights are saved

Graphical visualization of accuracy






BiLSTM Model: Training and Validation Accuracy
Training_Accuracy
Validation_Accuracy
Model:
0.95
0.90
0.85
0.80
0.75
0
5
10
15
20
Number of epochs


BiLSTM Model: Training and Validation Loss
Training_Loss
Validation_Loss
Model:
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
5
10
15
20
Number of epochs

📌 Future Improvements

- Use **pre-trained embeddings** like GloVe or Word2Vec for better text representation
- Upgrade to **Transformer models** (BERT, RoBERTa) for higher accuracy
- Add multi-class classification for **different types of spam**
- Deploy as a **web app or API** for real-time email/SMS spam detection

👩‍💻 Author

Arfa
BS Artificial Intelligence Student